



Universidad del Valle de Guatemala

Facultad de Ingeniería

Catedrático: Javier Fong

Sección: 20

Deep Learning y Sistemas Inteligentes – 2026

Laboratorio 1

Entrenamiento de Redes Neuronales

Integrantes

Joel Jaquez Carné: 23369

Guatemala, 2 de agosto del 2026

Repositorio del laboratorio:

https://github.com/jaq23369/Laboratorio1_Deep.git

1. Investigación: capas de PyTorch y optimizadores

1.1. Capas de red

Capa	Propósito	Parámetros relevantes
<code>nn.Linear</code>	Capa fully connected: $y = xW^T + b$. W, b son los pesos que la red ajusta durante el entrenamiento.	<code>in_features</code> , <code>out_features</code> , <code>bias</code> (default <code>True</code>)
<code>nn.ReLU</code>	$\max(0, x)$. Barata de calcular, activación por defecto en capas ocultas. Riesgo: neuronas “muertas”.	<code>inplace</code> (default <code>False</code>)
<code>nn.LeakyReLU</code>	Como ReLU pero deja pasar los negativos escalados por una pendiente pequeña, evitando neuronas muertas.	<code>negative_slope</code> (default 0,01)
<code>nn.Tanh</code>	Comprime la entrada a $(-1, 1)$; curva suave centrada en 0, se satura en valores extremos.	Ninguno relevante
<code>nn.Dropout</code>	Regularización: apaga neuronas al azar (prob. p) en entrenamiento; actúa como identidad en evaluación.	<code>p</code> (default 0,5), <code>inplace</code>
<code>nn.BatchNorm1d</code>	Normaliza (media 0, var 1) la salida de una capa oculta por mini-batch; estabiliza el entrenamiento.	<code>num_features</code> , <code>eps</code> , <code>momentum</code> , <code>affine</code>

1.2. Funciones de pérdida para regresión

Función	Propósito	Parámetros
<code>nn.MSELoss</code>	$(\hat{y} - y)^2$ promediado. Muy sensible a outliers por elevar al cuadrado.	<code>reduction</code>
<code>nn.L1Loss</code>	$ \hat{y} - y $ promediado. Más robusta a outliers; gradiente constante.	<code>reduction</code>
<code>nn.SmoothL1Loss</code>	Cuadrática si el error es menor a β , lineal si es mayor. Combina lo mejor de las dos anteriores.	<code>beta</code> (default 1,0), <code>reduction</code>

1.3. Optimizadores

lr: tamaño del paso de actualización de los pesos; muy alto desestabiliza, muy bajo hace lento el entrenamiento. **weight_decay**: regularización L2, encoge los pesos hacia 0 en cada paso para evitar overfitting.

Optimizador	Función en el entrenamiento del MLP y parámetros propios	Qué lo diferencia
SGD	Actualiza cada peso con $lr \cdot$ gradiente. momentum (0), nesterov (False), dampening (0).	No adapta el paso por peso; el más simple, necesita más tuning manual.
Adam	Promedios móviles del gradiente y su cuadrado por cada peso (betas = (0,9,0,999)), eps = 10^{-8} , amsgrad .	Converge rápido y robusto sin mucho tuning; combina adaptación + momentum.
RMSprop	Promedio móvil solo del gradiente al cuadrado (alpha = 0,99), eps , momentum , centered .	Adapta el paso por peso como Adam, pero sin memoria de la dirección del gradiente.

Fuentes: (documentación oficial de PyTorch): nn.Linear, nn.ReLU, nn.LeakyReLU, nn.Tanh, nn.Dropout, nn.BatchNorm1d, nn.MSELoss, nn.L1Loss, nn.SmoothL1Loss, optim.SGD, optim.Adam, optim.RMSprop.

2. Resultados de las iteraciones (Random Search)

#	Arquitectura	Act.	Optim. / LR	Batch/Ep	Regularización	MSE ($\times 10^9$)	MAE	RMSE
1	[32, 64, 16]	ReLU	Adam / 0.0011	128/80	L2=3.1e-5	2.67	\$34.2k	\$51.7k
2	[256, 16]	LReLU	Adam / 0.0031	256/50	Dropout .18 + L2=1.6e-5	2.68	\$34.9k	\$51.8k
3	[32]	Tanh	RMSprop / 0.0028	256/50	L1=1.2e-5	2.91	\$37.0k	\$54.0k
4	[32]	Tanh	RMSprop / 0.0007	128/50	L1=2.7e-6	2.93	\$37.6k	\$54.1k
5	[256]	ReLU	RMSprop / 0.0006	256/80	L2=1.9e-3	2.94	\$37.4k	\$54.2k
6	[64, 256]	ReLU	RMSprop / 0.0008	16/30	L2=1.6e-4	2.96	\$37.4k	\$54.4k
7	[256, 128, 32]	LReLU	RMSprop / 0.0006	16/30	L2=6.8e-5	3.00	\$37.3k	\$54.8k
8	[32, 64]	Tanh	RMSprop / 0.0056	256/80	Dropout .17	3.02	\$37.2k	\$54.9k
9	[64]	ReLU	Adam / 0.0011	64/50	Dropout .21 + L2=1.4e-5	3.14	\$39.0k	\$56.0k
10	[16, 16, 64]	ReLU	Adam / 0.0002	16/80	Dropout .13 + L2=1.8e-4	3.14	\$40.7k	\$56.0k
11	[256, 16]	LReLU	SGD / 0.0041	128/80	Dropout .44 + L2=1.1e-5	3.68	\$43.8k	\$60.7k
12	[16]	ReLU	Adam / 0.0023	16/80	Dropout .39	3.75	\$43.5k	\$61.2k
13	[16, 256, 64]	Tanh	SGD / 0.0002	128/30	L2=8.2e-3	4.59	\$49.5k	\$67.8k
14	[256, 16, 256]	LReLU	SGD / 0.0001	64/50	Dropout .45	5.15	\$52.7k	\$71.8k
15	[256]	LReLU	Adam / 0.0110	16/30	Dropout .42	5.21	\$49.4k	\$72.2k

Nota: LReLU = LeakyReLU. Todas las métricas (MSE, MAE, RMSE) calculadas sobre el conjunto de validación. La configuración #1 (Random 6) fue la mejor y se evaluó una única vez sobre test: MSE $\approx 2,89 \times 10^9$, MAE $\approx$ \$35,905, RMSE $\approx$ \$53,749.

3. Discusión y análisis

¿Qué cambio de hiperparámetro tuvo el mayor impacto positivo en las métricas de validación? ¿Y el mayor impacto negativo?

El mayor impacto positivo lo tuvo usar un buen optimizador con un `lr` bien calibrado. Las 8 mejores configuraciones (RMSE entre \$51,715 y \$54,922) usan Adam o RMSprop con `lr` entre 0.0006 y 0.0056, junto con muy poca regularización L1 o L2 (del orden de $1e-5$). El mayor impacto negativo lo tuvo un `lr` mal calibrado combinado con pocos epochs. Las 3 peores configuraciones (RMSE entre \$67,760 y \$72,192) usan SGD con `lr` muy bajo (0.0001 a 0.0002, insuficiente para converger en 30 o 50 epochs) o Adam con un `lr` algo alto (0.011) junto con dropout fuerte (0.42) y batch pequeño (16). La diferencia entre la mejor y la peor configuración es de casi \$20,500 en RMSE, casi un 40 % peor. En ambos casos el factor común fue el optimizador y el `lr`, más que la arquitectura.

¿Observó overfitting o underfitting en alguna de sus iteraciones? ¿Cómo lo identificó y qué hizo para mitigarlo?

Sí observé ambos casos, y los identifiqué mirando las curvas de pérdida. En la mejor configuración (Random 6) hay un indicio leve de overfitting, que es cuando el modelo aprende demasiado bien los datos de entrenamiento y empieza a perder capacidad de generalizar a datos nuevos. Se ve porque el train loss sigue bajando hasta el final (de 0.32 a 0.17) mientras el val loss se estanca cerca de 0.20 desde el epoch 40. El gap se queda pequeño porque esta configuración tiene una regularización L2 ligera (`weight_decay=3.1e-5`) que ayuda a contenerlo. En cambio, la peor configuración (Random 11) muestra underfitting, que es cuando el modelo ni siquiera logra aprender bien los datos de entrenamiento. Ahí el train loss y el val loss se mueven juntos todo el tiempo, con subidas y bajadas grandes entre 0.30 y 0.48, sin bajar de forma consistente. La causa fue la combinación de un batch muy pequeño (16, que hace que cada paso de entrenamiento sea más ruidoso), dropout fuerte (0.42) y solo 30 epochs. En el experimento completo, las configuraciones con batch más grande (128 o 256) y regularización más ligera evitaron este problema.

¿La regularización (L1, L2 o dropout) mejoró el desempeño en validación? ¿Cuál funcionó mejor para este dataset y por qué cree que fue así?

Sí mejoró el desempeño, pero solo en dosis pequeñas. Las 4 mejores configuraciones usan L2 (`weight_decay` entre $1.6e-5$ y $3.1e-5$) o L1 (`l1_lambda` entre $2.7e-6$ y $1.2e-5$), ambos con valores muy chicos. En cambio, dropout fuerte (mayor a 0.4) o un L2 grande ($8.2e-3$) aparecen en 4 de las 5 peores configuraciones. Creo que esto pasa porque mi red es pequeña (máximo 3 capas, hasta 256 neuronas) y el set de entrenamiento tiene 14,448 filas. No es un modelo con tanta capacidad como para sobreajustarse fuertemente, entonces una regularización agresiva no resuelve un problema que no es tan grande, y en cambio le quita al modelo capacidad de aprender el patrón real, sobre todo el dropout, que apaga neuronas de una red que ya es chica.

¿Cómo se relacionan el batch size y el número de epochs con la estabilidad y velocidad del entrenamiento que observó?

Se ve bastante claro comparando las curvas. Con batch de 16 (Random 7 y Random 11) las curvas salen muy ruidosas, con muchos picos, porque cada actualización de los pesos se calcula usando el gradiente de solo 16 ejemplos, una muestra chica que cambia mucho de un batch a otro. Con batch de 128 (Random 6, la mejor configuración) la curva es mucho más suave y estable. Sobre los epochs, Random 6 usa 80 y eso le da tiempo suficiente para converger

bien. Las configuraciones con SGD y solo 30 epochs nunca llegan a converger. En general, un batch más grande da gradientes más estables en cada paso, pero implica menos actualizaciones por epoch, así que conviene combinarlo con más epochs, que es justo el patrón de la mejor configuración.

Comparando MSE, MAE y RMSE, ¿qué le dice cada métrica sobre el comportamiento de su modelo que las otras no muestran?

El MSE (cerca de 2.89 mil millones en el modelo final sobre test) no se puede interpretar directamente porque queda en dólares al cuadrado, solo sirve como referencia de que penaliza fuerte los errores grandes. El MAE (\$35,905) da el error típico, en promedio el modelo se equivoca por unos \$36 mil. El RMSE (\$53,749) sale más alto que el MAE, y esa diferencia de casi \$17,800 dice algo que las otras dos no muestran solas, hay distritos donde el modelo falla mucho más que en el resto, porque el RMSE castiga esos errores grandes al elevarlos al cuadrado y el MAE no tanto. Esto coincide con algo que ya había encontrado en la exploración de datos, el precio real queda truncado en \$500,001 en un 4.7% de los distritos, así que el modelo nunca puede acertarle bien a esos casos, y esos errores grandes probablemente son los que más suben el RMSE frente al MAE.

Si tuviera que entrenar un modelo de producción con este dataset, ¿qué arquitectura e hiperparámetros elegiría, y qué estrategia de búsqueda usaría para seguir optimizando? Justifique.

Partiría de la configuración ganadora (Random 6, arquitectura [32, 64, 16], ReLU, Adam con `lr` cerca de 0.0011, batch de 128, 80 epochs, $L2=3.1e-5$), y le agregaría dos cosas. La primera es early stopping basado en el val loss, que es simplemente detener el entrenamiento cuando el val loss deja de mejorar, ya que en su propia curva se estanca desde el epoch 40 y seguir después de eso solo roza el overfitting leve que ya identifiqué. La segunda es agregar más variables (feature engineering), como habitaciones por hogar o ingreso per cápita, porque varias de las features originales están agregadas a nivel de distrito y no por vivienda, lo que le pone un techo a cuánto puede aprender el modelo. Para esta primera ronda usé Random Search porque con 7 hiperparámetros un grid search completo crecería exponencialmente, con solo 3 valores por hiperparámetro ya serían cerca de 2,187 combinaciones. Para una siguiente ronda, ya con una región prometedora identificada (Adam, `lr` entre 0.001 y 0.003, $L2$ chico, batch entre 128 y 256), usaría optimización bayesiana, una estrategia que usa el resultado de cada iteración anterior para decidir qué probar en la siguiente, en vez de muestrear a ciegas como random search. Es más eficiente cuando cada entrenamiento sale caro, como pasaría en un modelo de producción con más datos y más épocas.